Contents

Documentación de Diagramas de Actividad — Aliflow 1
1. Autenticación 1
2. Recargar saldo 1
4. Retirar y entregar almuerzo 2
5. Sincronización con el ERP externo (patrón Outbox) 3
Procesos no cubiertos con diagrama de actividad propio (y por qué) 6
Documentación de Diagramas de Secuencia — Aliflow 7
1. Comprar almuerzo (el algoritmo central) 8
2. Retirar y entregar almuerzo (redención atómica del código) 8
3. Recargar saldo 9
4. Sincronización con el ERP externo (patrón Outbox) 9
Por qué estos 4 y no otros 10
Hallazgo transversal de esta ronda 10
Documentación de Diagramas de Estado — Aliflow 11
1. Orden 11
2. CodigoRetiro 12
3. Pago 12
4. EventoSincronizacion 12
5. Cartilla de fidelidad 14
Por qué estos 5 y no otros 16

Documentación de Diagramas de Actividad — Aliflow

Cinco diagramas de actividad, cubriendo los procesos del sistema con lógica de decisión real (ramas, excepciones, o relevancia transaccional). Cada uno usa carriles (swimlanes) por actor para dejar explícito quién hace qué.

---

1. Autenticación

Fuente: `../../uml/actividad-autenticacion.puml` · **Referencia:** `est-1`, UC1.

Cubre la rama que el flujo original ya identificaba como crítica (validación de dominio institucional) y la rama de "primer ingreso" (creación de perfil y tarjeta virtual).

2. Recargar saldo

Fuente: `../../uml/actividad-recarga-saldo.puml` · **Referencia:** `est-2`, UC2. ## 3. Comprar almuerzo

Fuente: `../../uml/actividad-compra-almuerzo.puml` · **Referencia:** `est-4`, UC4.

Es el proceso más crítico del sistema — incluye las dos validaciones que ya identificamos como riesgo (saldo insuficiente, stock agotado por concurrencia) y referencia explícitamente el mecanismo de corrección que agregamos en la revisión anterior: `Plato.reservarStock` con bloqueo

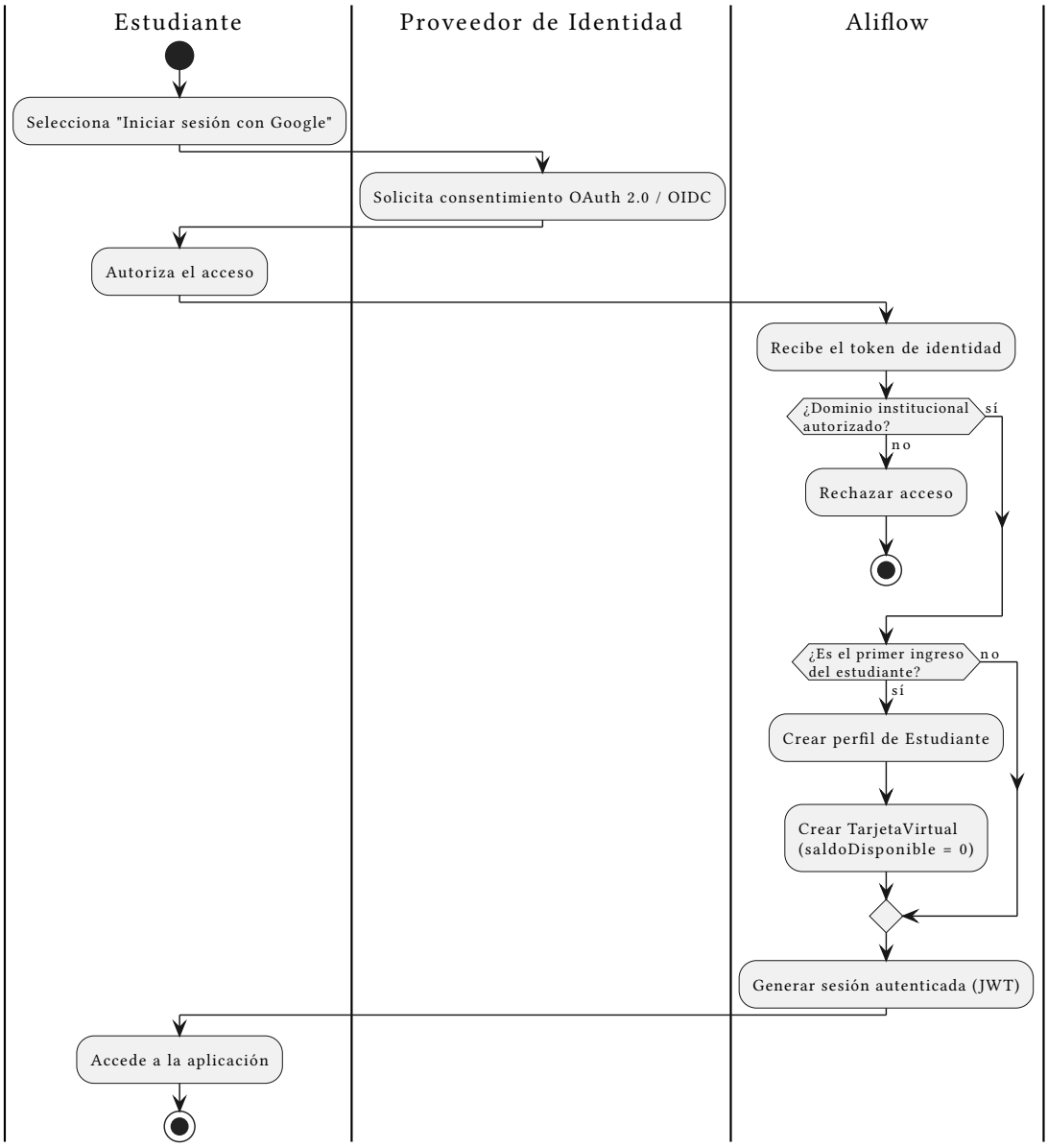


Figura 1: Actividad: autenticación

optimista (`version`), y el invariante de un solo proveedor por orden. También muestra que la orden se registra en `RegistroAuditoria`, cerrando el vacío que había detectado el riesgo R-09.

4. Retirar y entregar almuerzo

Fuente: `../../uml/actividad-retiro-entrega.puml` · **Referencia:** `est-6`, `op-2` a `op-5`, UC5.

Cubre las tres ramas de excepción que el flujo original ya documentaba: sin código (búsqueda manual), orden no encontrada, y orden ya entregada/código inválido.

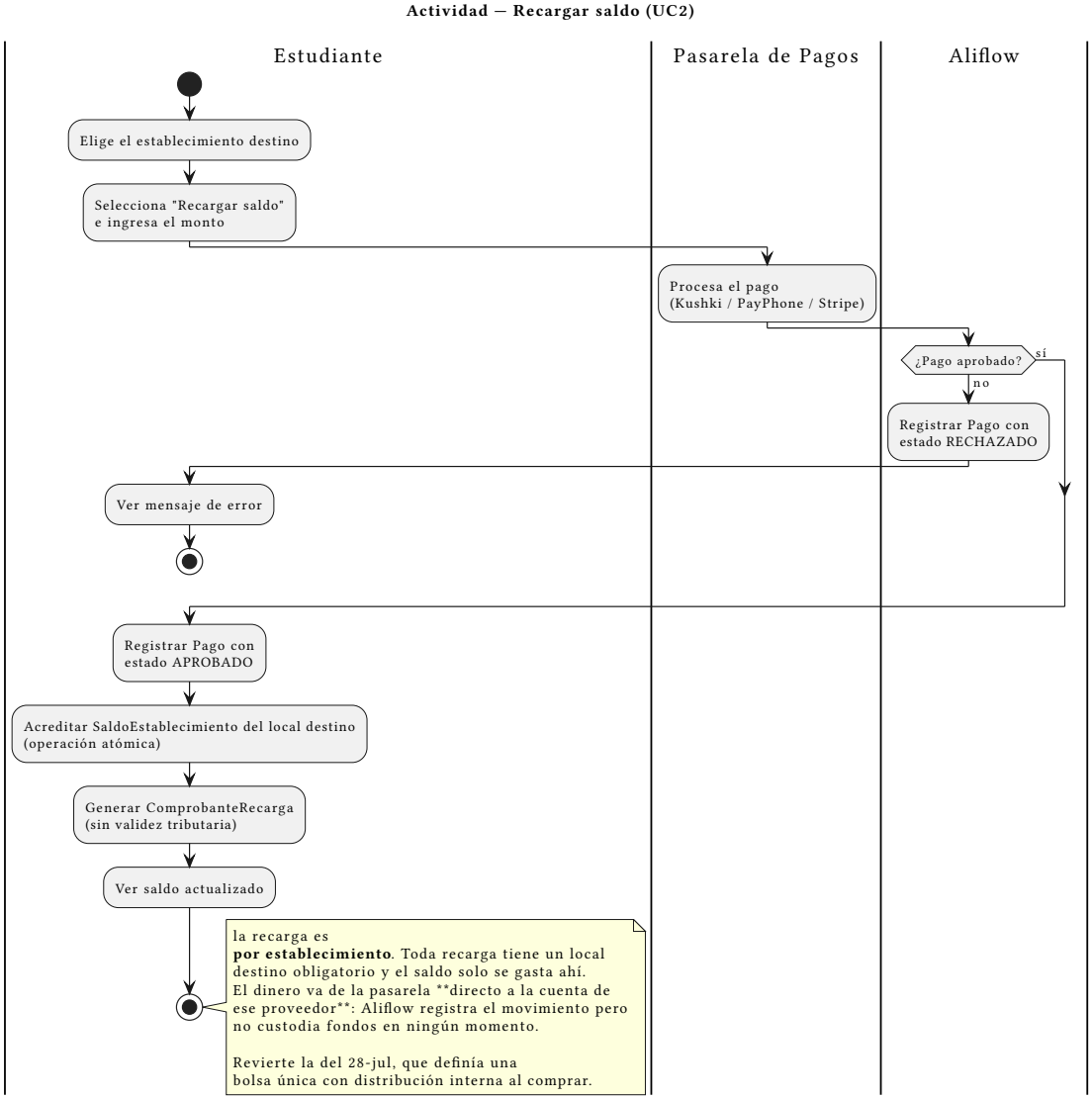


Figura 2: Actividad: recarga de saldo

Ampliado con la partición “Fidelidad” al final: si el local tiene programa activo y el estudiante no acumuló ya un sello ese día, la entrega acredita un sello y —si con eso se completa la cartilla— el estudiante ve el aviso de premio disponible. Está deliberadamente **después** de marcar la orden como ENTREGADO y de registrar la auditoría: el sello es una consecuencia de la entrega, no una condición para completarla. Si el módulo de fidelidad fallara, la entrega ya ocurrió y no se revierte.

5. Sincronización con el ERP externo (patrón Outbox)

Fuente: `../../uml/actividad-sincronizacion-erp.puml` · **Referencia:** `../../../Hallazgos-Ingenieria-API-Generica.md` sección 3.4, `../../uml/objeto-integracion-erp.puml`.

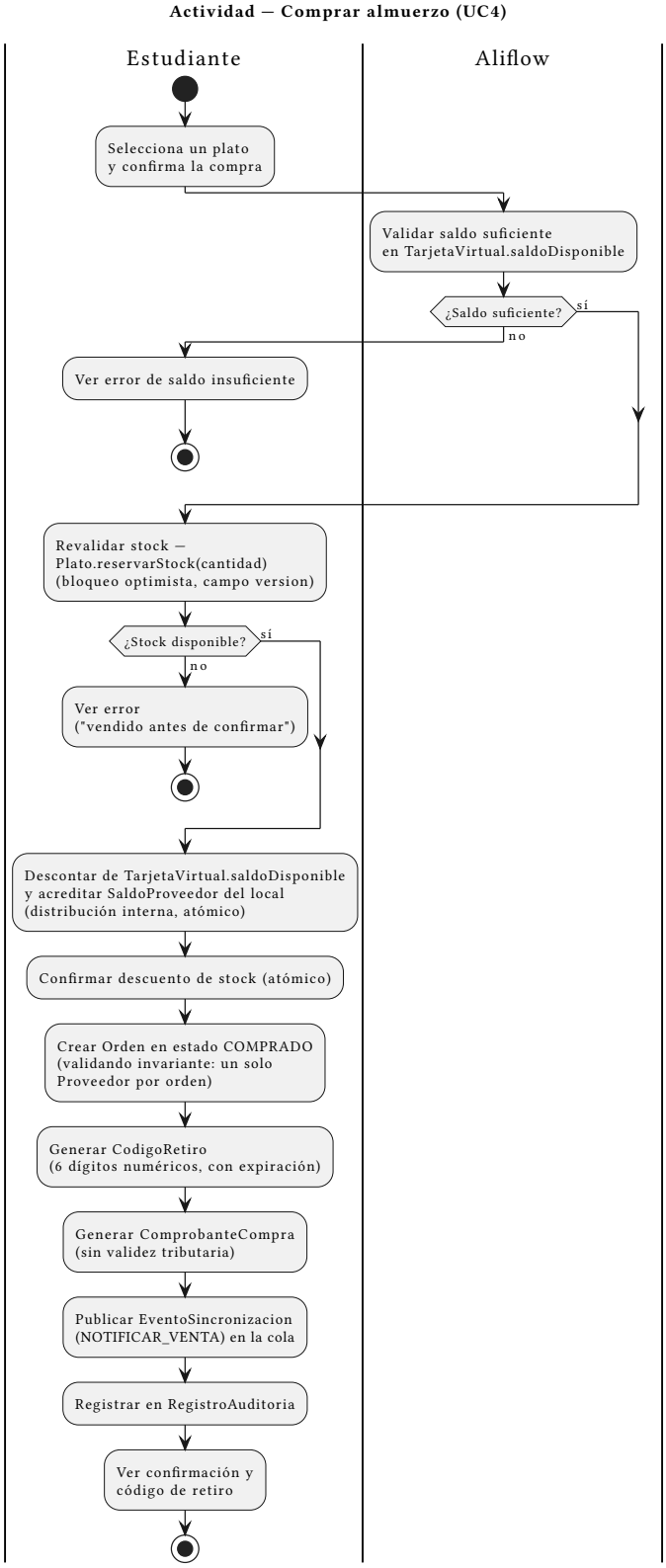


Figura 3: Actividad: comprar almuerzo

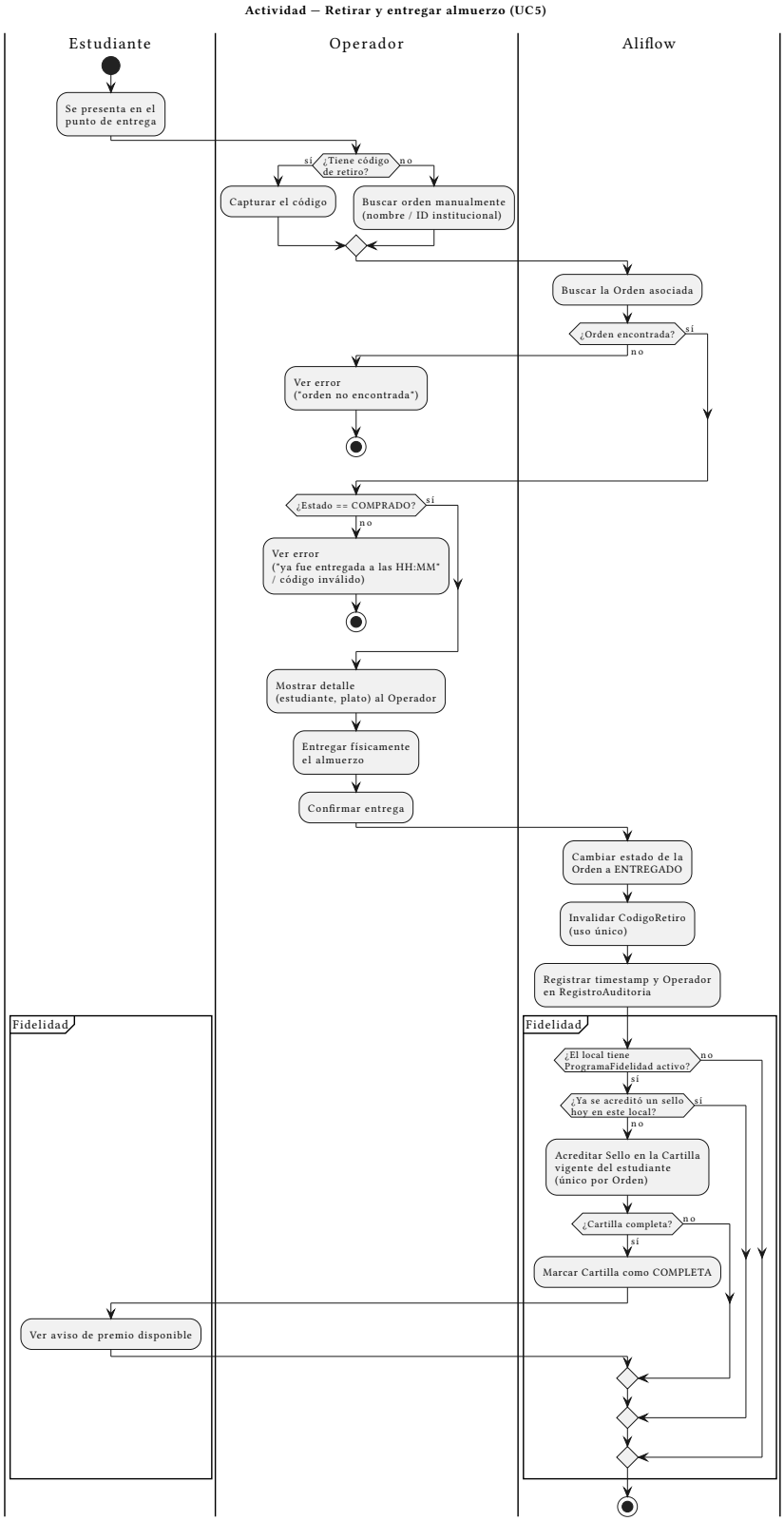


Figura 4: Actividad: retiro y entrega

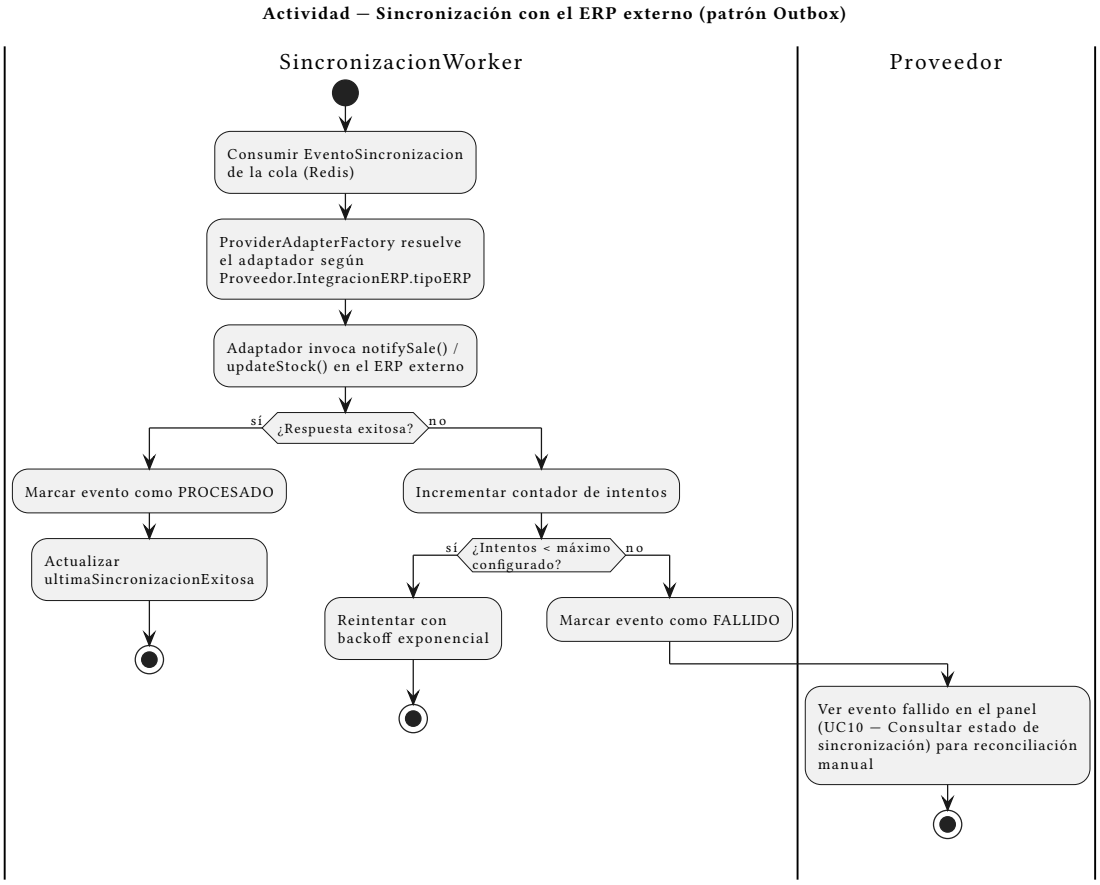


Figura 5: Actividad: sincronización ERP

Este es el proceso que materializa el patrón Outbox: consumir el evento, invocar el adaptador correspondiente, y — si falla — reintentar con backoff hasta un máximo, marcando el evento como `FALLIDO` para reconciliación manual en el panel del Proveedor (UC10). Es la misma secuencia que ya ilustramos con datos concretos en el diagrama de objetos (`../../uml/objeto-integracion-erp.puml`, evento fallido tras 3 intentos contra Alpwin en Caramel Coffee, junto a uno exitoso contra Contífico en Barú) — aquí se ve el proceso general, allá el caso puntual.

Procesos no cubiertos con diagrama de actividad propio (y por qué)

- **Consultar menú (UC3)** y **Administrar menú (UC8)** — son mayormente lineales (sin ramas de decisión relevantes más allá de la sincronización con el ERP, ya cubierta en el diagrama 5); no se justificaba un diagrama separado.
- **Configurar integración (UC7)**, **Consultar métricas (UC9/UC13)**, **Consultar estado de sincronización (UC10)**, **Consultar detalle de venta (UC11)** — son consultas/configuración sin lógica de negocio compleja.

- **UC12 (Gestionar usuarios del local)** y **UC14 (Configurar programa de fidelidad)** — son CRUD de configuración sin ramas de decisión interesantes; no aportan como diagrama de actividad.
- **UC15 (Canjear premio)** — reutiliza casi por completo el flujo de UC4 (compra) con total $0; en vez de duplicar el diagrama, la diferencia está documentada en `../02-Modelamiento-Parte-Estatica/a-Casos-de-Uso.md` y en el diagrama de estado de la cartilla. Los antiguos UC13/UC14 desaparecieron junto con el rol de super-admin descartado por el cliente.

Documentación de Diagramas de Secuencia — Aliflow

Cuatro diagramas de secuencia, cubriendo los **algoritmos transaccionales relevantes** del sistema: los que mueven dinero, cambian estado de forma atómica, o dependen de un sistema externo con posibilidad de fallo.

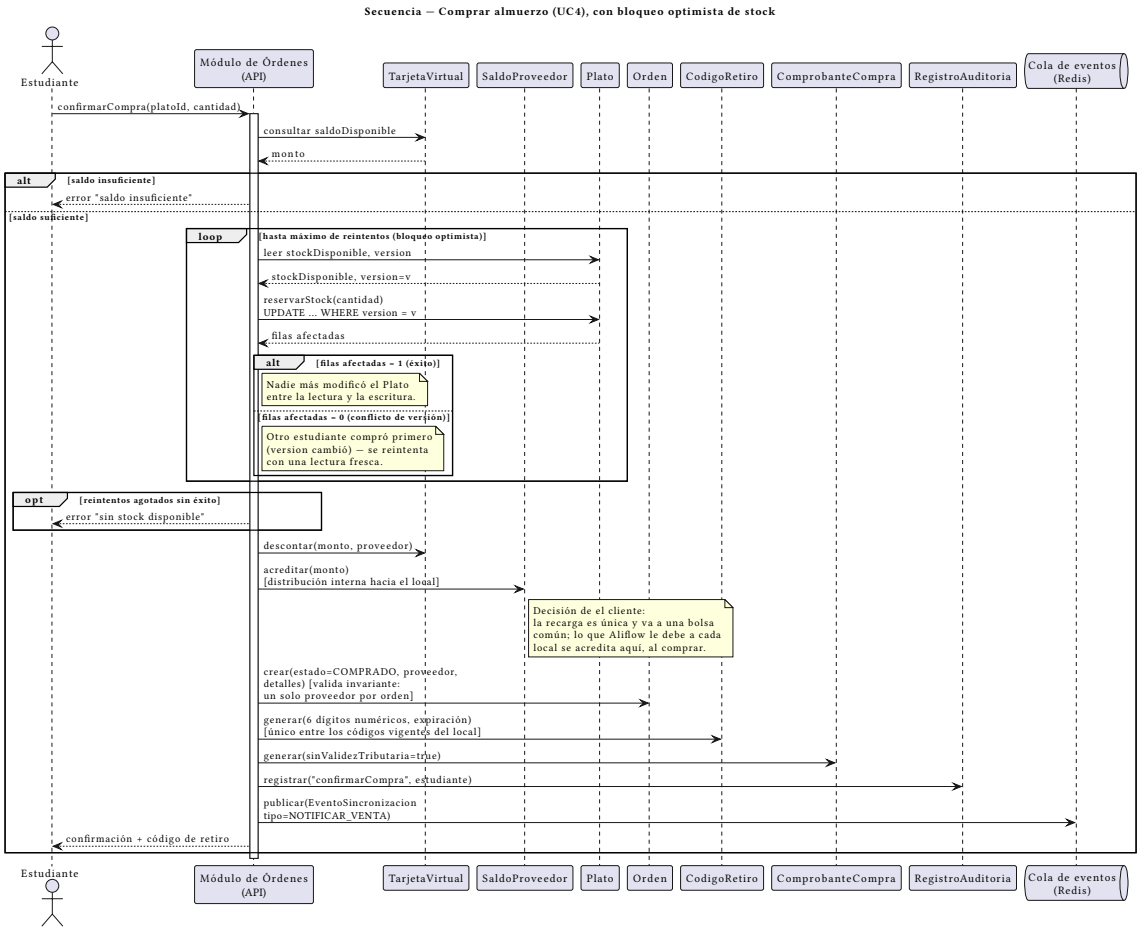


Figura 6: Secuencia: comprar almuerzo

1. Comprar almuerzo (el algoritmo central)

Fuente: `../../uml/secuencia-compra-almuerzo.puml` · **Referencia:** UC4, `../../uml/actividad-compra-almuerzo.puml`.

Este es el diagrama que **resuelve en detalle** el control de concurrencia que quedó pendiente en el diagrama de clases (`Plato.version` / `reservarStock`, agregado en la revisión anterior): el `loop` muestra explícitamente que la actualización de stock es un `UPDATE... WHERE version = v`, y que si otro estudiante ya compró primero (0 filas afectadas), se reintenta con una lectura fresca en vez de fallar o, peor, descontar stock que ya no existe. Esto es lo que evita la venta duplicada de la última unidad — un riesgo que estaba documentado en el flujo original desde el principio pero que no tenía, hasta ahora, un mecanismo concreto mostrado en ningún diagrama.

2. Retirar y entregar almuerzo (redención atómica del código)

Fuente: `../../uml/secuencia-retiro-entrega.puml` · **Referencia:** UC5.

Hallazgo nuevo de esta revisión, no documentado antes: el mismo tipo de condición de carrera que existe en el stock (dos compras simultáneas por la última unidad) también puede ocurrir aquí — dos Operadores validando el mismo código casi al mismo tiempo. Se resuelve con el mismo principio (`UPDATE... WHERE usado = false`, atómico a nivel de base de datos): si la

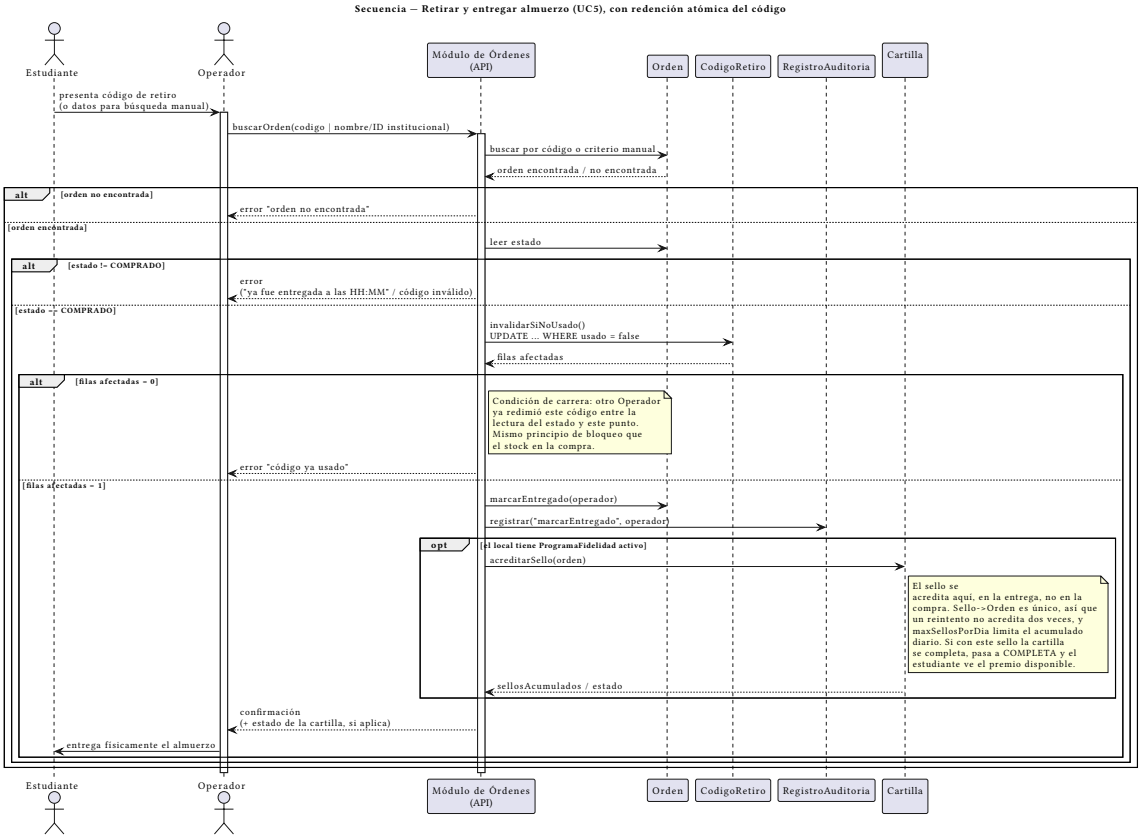


Figura 7: Secuencia: retiro y entrega

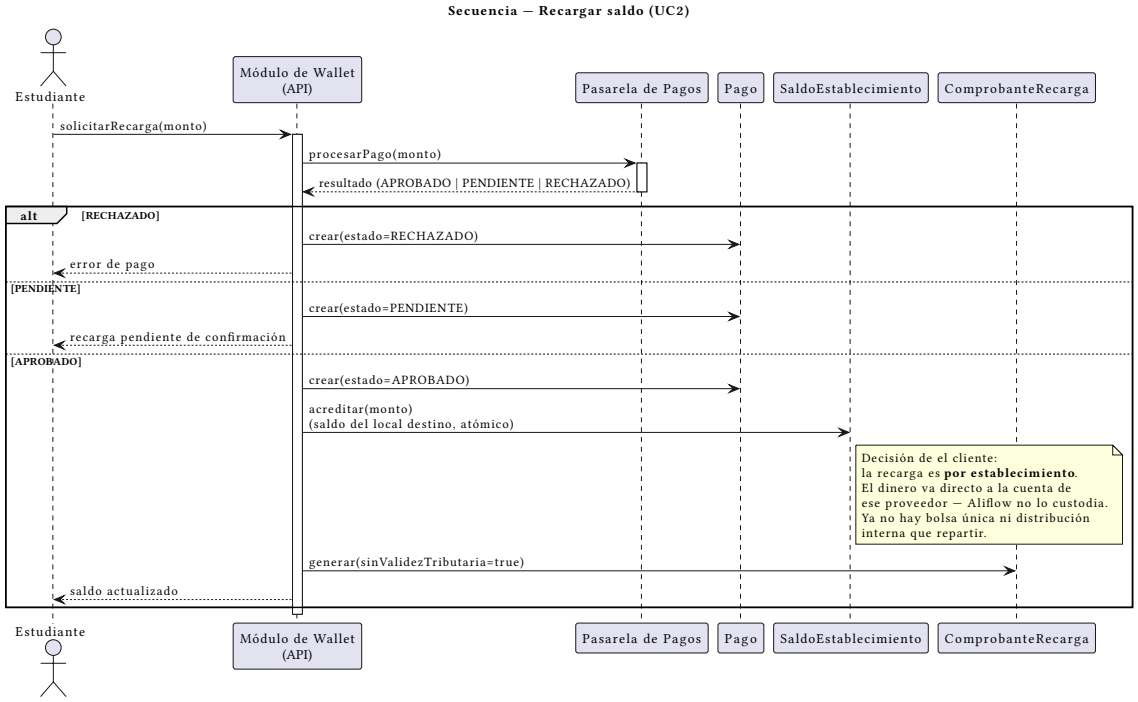


Figura 8: Secuencia: recarga de saldo

actualización afecta 0 filas, significa que alguien más ya redimió el código en el intervalo, y se informa como error en vez de marcar una segunda entrega. No hace falta ningún campo nuevo en `CodigoRetiro` (el booleano `usado` ya alcanza), solo que la actualización sea atómica y condicional.

Ampliado con el bloque `opt` de acreditación del sello de la cartilla de fidelidad, que ocurre después de `marcarEntregado`. Va dentro de un `opt` y no de la rama principal porque el programa de fidelidad es **opcional por local**: si el local no lo tiene activo, el flujo de entrega es idéntico al de antes.

3. Recargar saldo

Fuente: `../../uml/secuencia-recarga-saldo.puml` · **Referencia:** UC2.

Incluye la rama `PENDIENTE` del pago (no solo APROBADO/RECHAZADO), consistente con que las pasarelas candidatas (Kushki/PayPhone/Stripe, ver `../../uml/diagrama-componentes.puml`) suelen tener estados intermedios asíncronos. **Actualizado:** con la decisión de el cliente de recarga única, el paso de distribución desapareció de esta secuencia — el pago aprobado acredita directamente `TarjetaVirtual.saldoDisponible` y se emite el comprobante. La acreditación al local se movió a `../../uml/secuencia-compra-almuerzo.puml`.

4. Sincronización con el ERP externo (patrón Outbox)

Fuente: `../../uml/secuencia-sincronizacion-erp.puml` · **Referencia:** `../../../Hallazgos-Ingenieria-API-Generica.md` sección 3.4, `../../uml/objeto-integracion-erp.puml`, `../../uml/actividad-sincronizacion-erp.puml`.

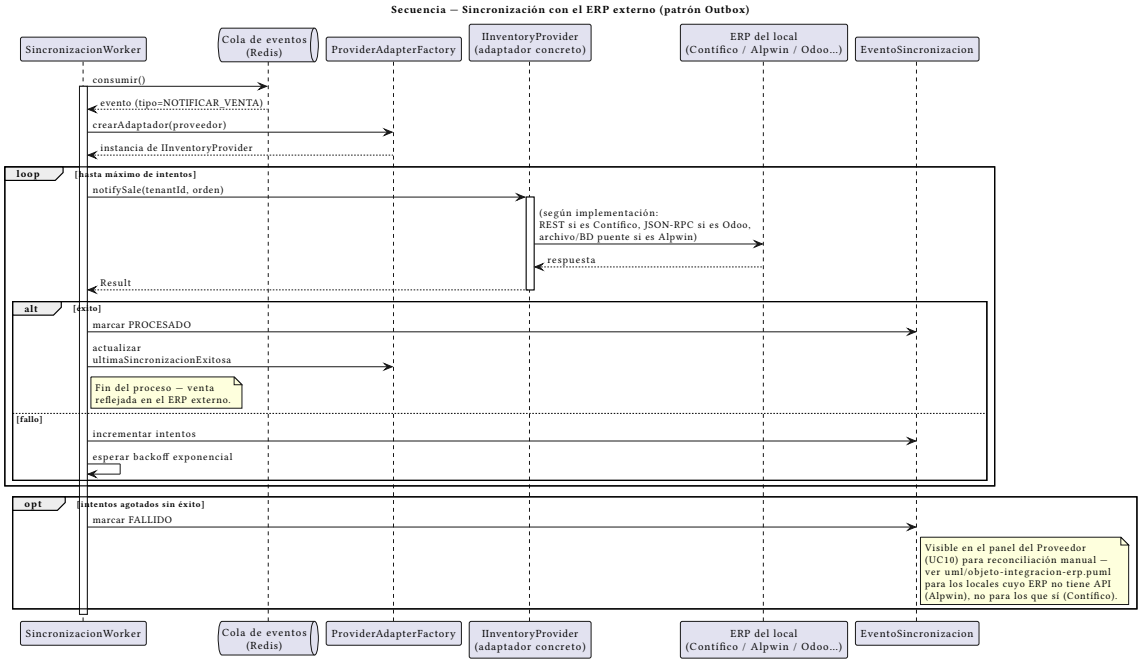


Figura 9: Secuencia: sincronización ERP

Muestra el mismo proceso que el diagrama de actividad correspondiente, pero a nivel de mensajes entre objetos concretos — útil para ver exactamente qué le llega a `IInventoryProvider` (`notifySale(tenantId, orden)`) y qué determina si el evento se reintenta o se marca `FALLIDO`.

Por qué estos 4 y no otros

Los demás procesos (consultar menú, administrar menú, configurar integración, consultar métricas) no involucran atomicidad, dinero, ni un sistema externo que pueda fallar a mitad de camino — son operaciones de lectura o escritura simple, sin el tipo de complejidad que un diagrama de secuencia está pensado para exponer. Se cubrieron en cambio con el diagrama de actividad correspondiente cuando tenían alguna rama de decisión relevante.

Hallazgo transversal de esta ronda

Al diseñar el diagrama de retiro, se detectó una condición de carrera (doble redención del código) que no estaba identificada en ningún documento anterior — ni en el flujo original, ni en `../01-Especificacion-de-Requerimientos/06-Gestion-de-Riesgos.md`. Vale la pena agregarla formalmente al registro de riesgos como un riesgo más (probabilidad baja, dado que requiere dos Operadores validando el mismo código en el mismo instante, pero impacto real si ocurre: dos entregas físicas de un mismo almuerzo).

Documentación de Diagramas de Estado — Aliflow

Cinco diagramas de estado, para los objetos del sistema con un ciclo de vida real (más de un estado posible y transiciones con condiciones).

---

1. Orden

Fuente: `../../uml/estado-orden.puml` · **Referencia:** UC4/UC5, `../../uml/diagrama-clases.puml`.

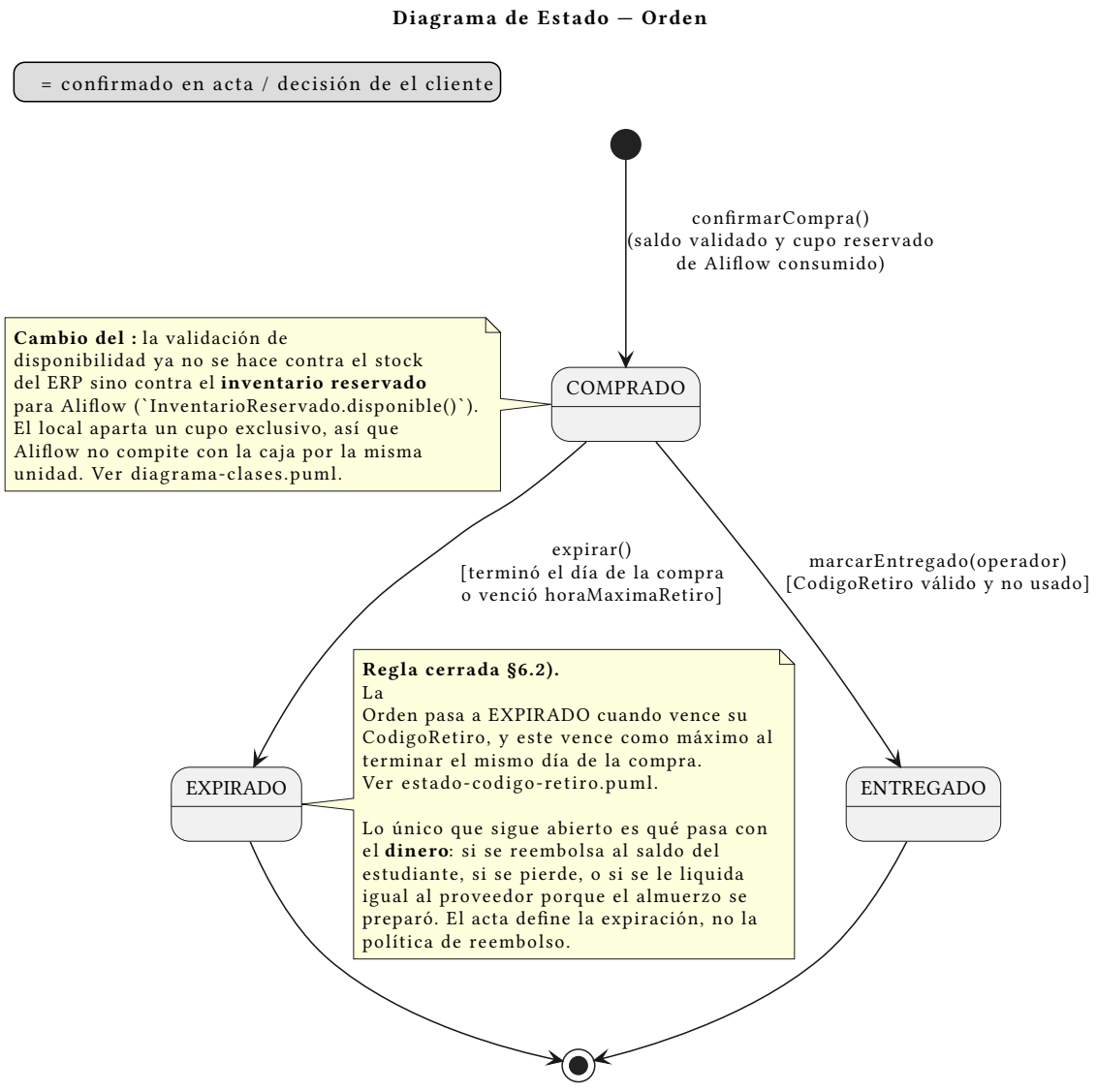


Figura 10: Estado: Orden

`COMPRADO → ENTREGADO` es el camino confirmado del flujo original. `COMPRADO → EXPIRADO` ocurre cuando vence el `CodigoRetiro` asociado. **Lo único que sigue abierto** es qué pasa con el **dinero** de una orden vencida: si se reembolsa, se pierde, o se le liquida al proveedor porque el almuerzo se preparó. El acta define la expiración, no la política de reembolso.

2. CodigoRetiro

Fuente: `../../uml/estado-codigo-retiro.puml` · **Referencia:** UC5, `../../uml/secuencia-retiro-entrega.puml`. | Estado | Significado | |—|—| | **VÁLIDO** | El pedido está disponible para ser retirado | | **UTILIZADO** | El Operador confirmó la entrega | | **VENCIDO** | Terminó el horario de retiro o terminó el día de la compra |

`VÁLIDO → UTILIZADO` es la transición atómica y condicional (`UPDATE... WHERE estado='VALIDO'`) que resuelve el riesgo R-14 (doble redención). `VÁLIDO → VENCIDO` dispara además la expiración de la `Orden` asociada — son dos objetos, pero una sola regla de negocio.

Regla de vigencia: el código vale **únicamente durante el día en que se hizo la compra**. `fechaExpiracion` se calcula como el mínimo entre la `horaMaximaRetiro` que configuró el local y el fin de ese día. Si se presenta otro día, debe mostrarse como vencido. Esto **cierra la decisión #5**, abierta desde el 27-jul.

Un efecto colateral que conviene notar: acotar la vigencia a un solo día reduce mucho el universo de códigos vigentes simultáneamente. Eso es justamente lo que hace viable exigir unicidad con solo 6 dígitos, y **refuerza la mitigación del riesgo R-15** sin que hubiera que hacer nada más.

Formato del código: es un **código numérico corto de 6 dígitos**, porque el estudiante se lo dicta de viva voz al Operador y este lo digita. El formato no altera la máquina de estados, pero trae dos consecuencias de diseño que quedaron anotadas en el diagrama:

1. **Unicidad acotada, no global.** Seis dígitos son ~1 millón de combinaciones: alcanzan de sobra si la unicidad se exige solo entre los códigos **Vigentes de un mismo local**, pero no si se pretende que sean únicos históricamente. La generación reintenta ante colisión.
2. **Espacio de búsqueda pequeño.** Un código de 6 dígitos es adivinable por fuerza bruta de una forma que un UUID no lo era. Se registró como riesgo **R-15** (`../01-Especificacion-de-Requerimientos/06-Gestion-de-Riesgos.md`), con la mitigación correspondiente.

3. Pago

Fuente: `../../uml/estado-pago.puml` · **Referencia:** UC2, `../../uml/secuencia-recarga-saldo.puml`.

Simple pero real: `PENDIENTE` no es solo un estado transitorio instantáneo — puede persistir si la pasarela de pagos responde de forma asíncrona (riesgo R-06, `../01-Especificacion-de-Requerimientos/06-Gestion-de-Riesgos.md`). Solo `APROBADO` genera una `Recarga`.

4. EventoSincronizacion

Fuente: `../../uml/estado-evento-sincronizacion.puml` · **Referencia:** `../../../Hallazgos-Ingenieria-API-Generica.md` sección 3.4.

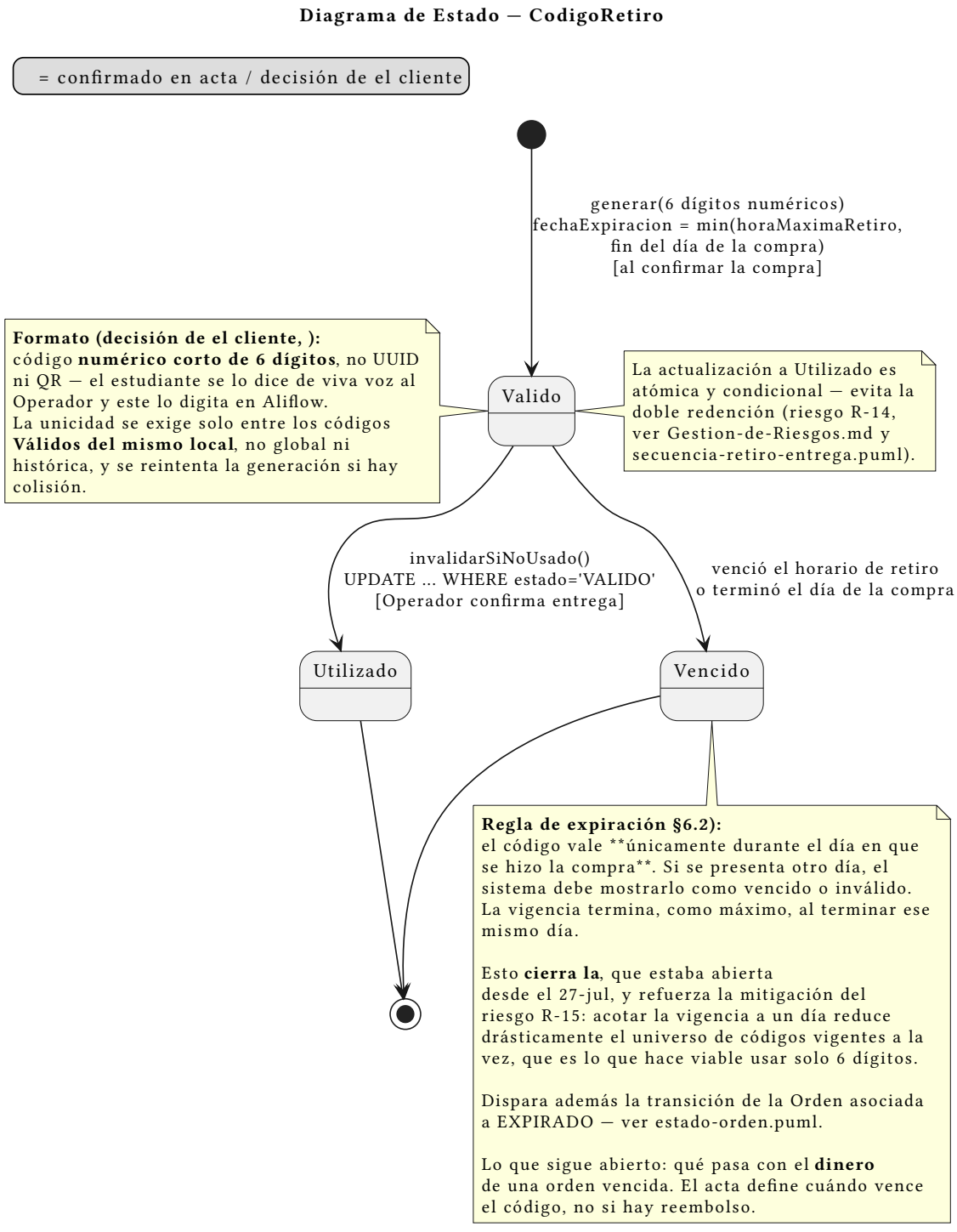


Figura 11: Estado: CodigoRetiro

El auto-loop en `PENDIENTE` (reintento con backoff) y la transición a `FALLIDO` tras agotar intentos son exactamente el comportamiento ya mostrado en el diagrama de secuencia y en el diagrama

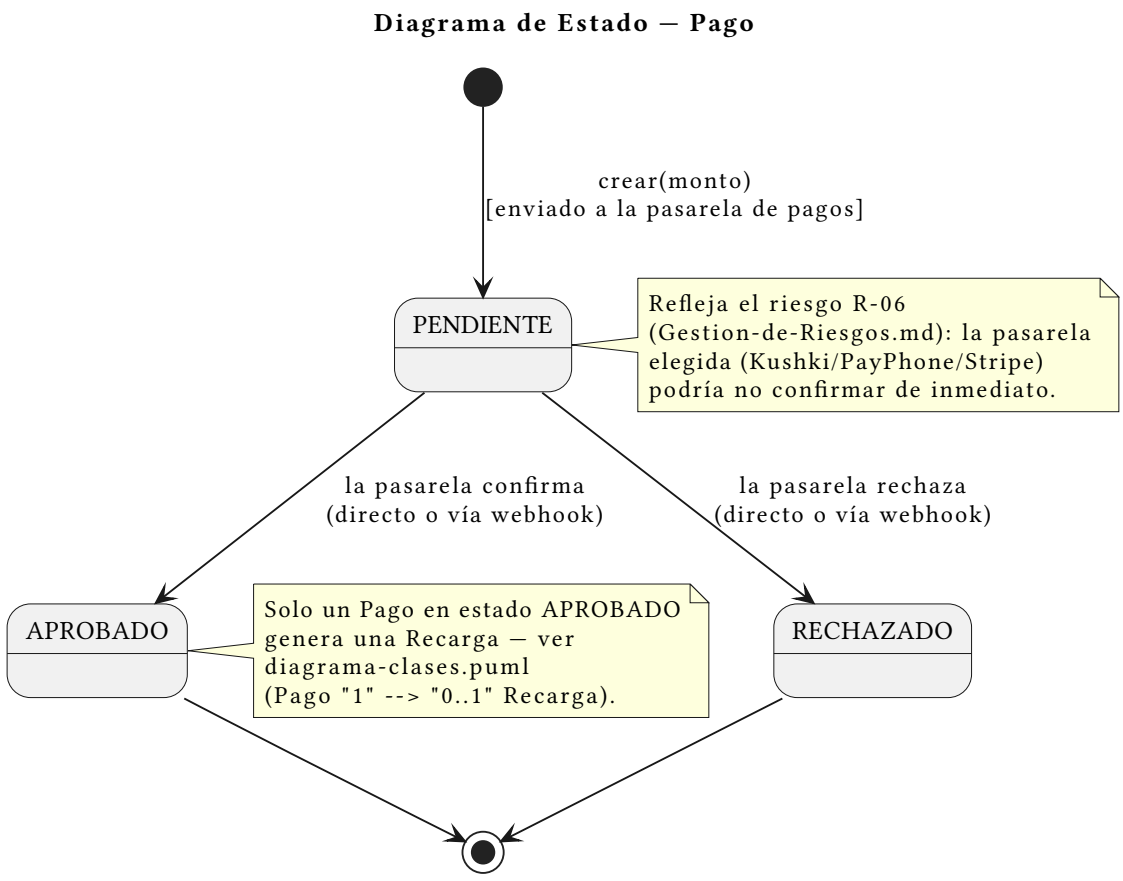


Figura 12: Estado: Pago

de objetos con el caso real de Alpwin en Caramel Coffee — este diagrama es la vista de ciclo de vida del mismo mecanismo.

5. Cartilla de fidelidad

Fuente: `../../uml/estado-cartilla.puml` · **Referencia:** UC13/UC14/UC15, `../../uml/diagrama-clases.puml` paquete "Fidelidad".

Requisito nuevo, todo en amarillo: la cartilla de fidelidad. Es el diagrama de estado que más aporta de los cinco, porque el ciclo de vida **es** la regla de negocio — cuándo suma un sello, cuándo se puede canjear y cuándo se pierde.

Cuatro estados: `EN_CURSO` (acumulando), `COMPLETA` (premio disponible), `CANJEADA` y `EXPIRADA`. Dos detalles que no son obvios y por eso están anotados en el propio diagrama:

- **El auto-loop en `EN_CURSO` se dispara desde `marcarEntregado`, no desde `confirmarCompra`.** Una orden comprada y nunca retirada no suma sello. Sin esta regla, la cartilla se puede llenar sin ir nunca a buscar el almuerzo, y el local termina regalando un premio por ventas que no ocurrieron físicamente.

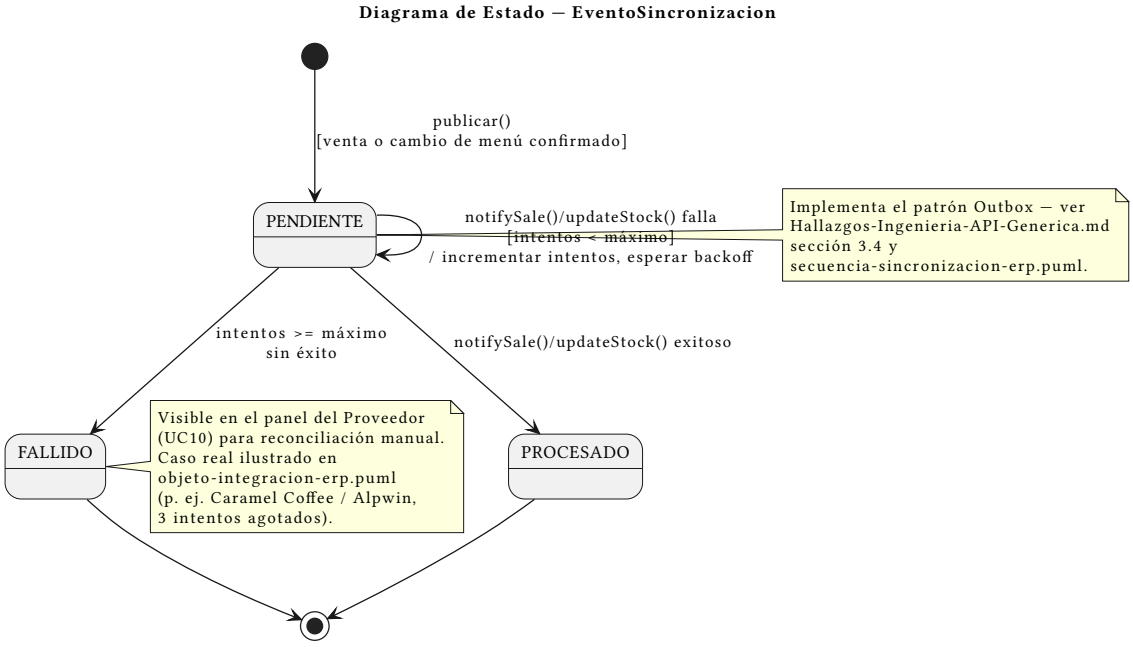


Figura 13: Estado: EventoSincronizacion

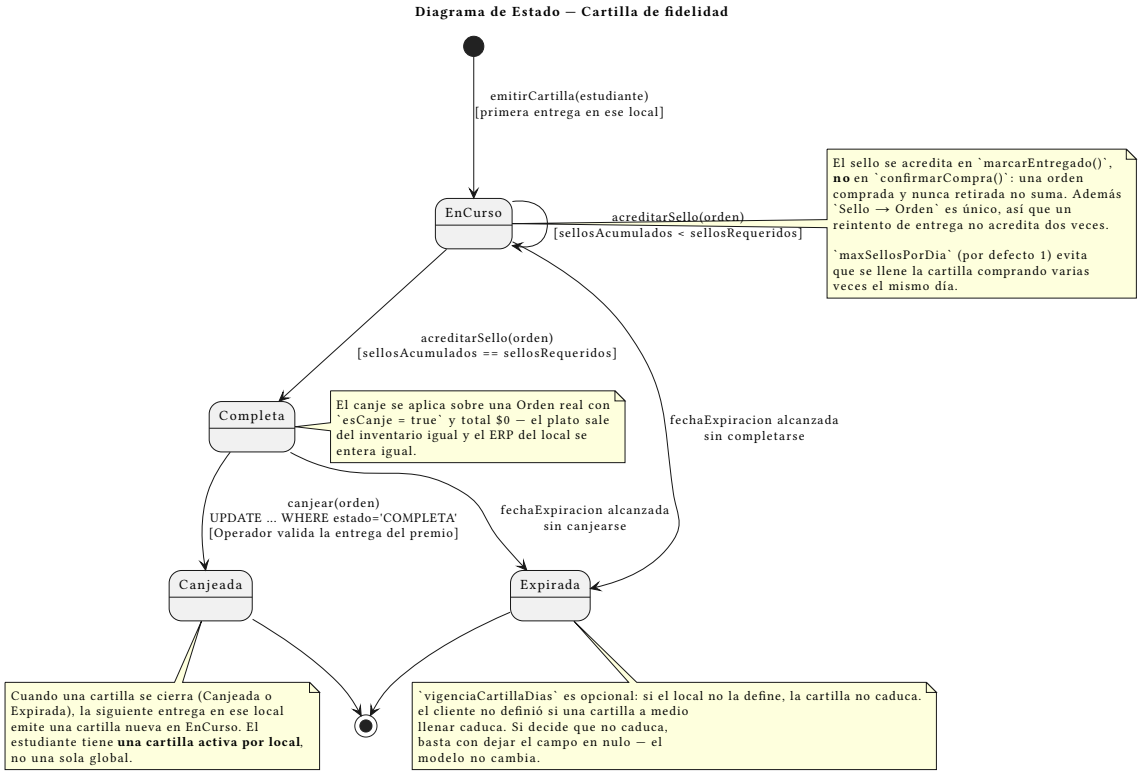


Figura 14: Estado: Cartilla

el mismo mecanismo que resuelve la doble redención del código de retiro (R-14). Sin esto, dos canjes simultáneos entregarían dos premios por una sola cartilla.

`EXPIRADA` es la parte más tentativa: el cliente no dijo si una cartilla a medio llenar caduca. Se modeló el estado con un campo `vigenciaCartillaDias` configurable; si la decisión es que no caduca, el campo queda nulo y la transición nunca se dispara — el modelo no cambia.

Por qué estos 5 y no otros

`Estudiante`, `Proveedor`, `Plato` y `ProgramaFidelidad` no tienen un ciclo de vida con estados discretos más allá de `activo`/`inactivo` (una sola transición booleana, no justifica un diagrama). `SaldoProveedor` y `TarjetaVirtual` cambian de valor (monto) pero no de "estado" en el sentido UML. Se priorizaron los objetos cuyo estado determina qué operaciones son válidas en cada momento.